AI–Based Code Review and Pull-request Assistant System with IDE Integration

Vaibhav Admane[1], Harshal Patil[2], Aditya Patil[3], Vaishnavi Agre[4], Assistant Professor Vishwajit Barbudhe[5], Associate Professor Dr. Atmeshkumar Patel[6]

Department of Artificial Intelligence and Data Science

Sandip Institute of Technology and Research Centre (SITRC)

Nashik, India

[1]vaibhavadmane03@gmail.com, [2]harshalpatil8482@gmail.com, [3]adityapatil091282@gmail.com, [4]agrevaishnavi835@gmail.com, [5]vishwajit.barbudhe@sitrc.org, [6]atmeshkumar.patel@sitrc.org

Abstract - Modern software development makes a significant use of code reviews in order to uphold the quality of the software, find defects in the code, and ensure that coding standards are met. However, traditional manual code reviews are often time-consuming, inconsistent, and susceptible to human intervention, which can have a considerable impact on delaying the development cycle and permitting bugs and security vulnerabilities to exist at later stages in the software development lifecycle. To overcome these issues this work presents an Artificial Intelligence based Code Review and Pull-Request Assistant integrated in the developer's coding environment. The proposed system combines rule-based static analysis and machine learning-based context-specific code understanding in order to provide automated and real-time feedback during development. Static analysis tools are used to catch syntax errors, code style violations, complexity problems, and potential security vulnerabilities, whereas transformer-based models are used to analyse code semantics to find deeper logical issues and make suggestions for improvement. A common analysis pipeline passes the results for both the layers through before actionable recommendations urgency is present and presented directly on the development interface. The system architecture ensures automated capture of bug, vulnerability and checking the code quality with less context switching for the coder. Experimental demonstrations demonstrate that the approach is quite capable of establishing common coding issues and providing shapeful feedback which is beneficial for more effective code dependability and efficient code review flows. By adding automated analysis into the development environment itself the proposed assistant helps to improve the productivity of developers and to achieve more consistent and scalable code review practices.

Keywords: Artificial Intelligence, Machine Learning, Automated Code Review, Static Code Analysis, Push Request Analysis, IDE Integration, Software Quality Assurance, CodeBERT, CodeT5, Developer Productivity

I. INTRODUCTION

The growth of the modern software systems has makes the software development processes complex and so the quality assurance of the code becomes crucial in the life-cycle of the development. Code reviews are crucial in the identification of defects, management of maintainability and enforcement of coding standards before the software is deployed. Traditionally, these reviews are done manually by developers who go through and provide feedback about changes to the code to ensure its correctness and quality. However, manual review processes often have a number of limitations such as inconsistency, time consumption, and reliance on individual expertise. As software projects grow in size and complexity, such limitations can cause feedback cycles can take a long time and defects can be missed. Automated tools have been introduced to help developers detect syntactic errors, violate coding style, and find potential vulnerabilities but many traditional tools rely on some, mostly, rule-based static analysis techniques. While these tools work good in identifying the simple issues, there is a context in the code semantics and developer intent that an AI-based tool may not be able to understand. The growing need for scalable and intelligent development tools has thus encouraged the development of research in automated systems that can offer more comprehensive code analysis and assistance to developers. Advances in artificial intelligence have opened a new opportunity for optimizing software engineering processes by creating systems that are capable of learning patterns from large-scale code repositories and making intelligent recommendations during the development of systems.

Artificial intelligence and machine learning methods have become the game-changing technologies in the campo of software engineering to allow for more sophisticated forms of automated analysis and support for developers. Machine learning models can be used to analyze datasets of thousands or millions of lines of source code to find patterns in source code that relate to defects, code smells, and best practices. By capitalizing on the ability to assist developers in spotting any potential issues earlier in the development cycle or, in any

case, increase the overall trustworthiness of the code, AI-controlled systems can play a role in. Natural language processing and the development of transformer-based architectures have added even more potential to AI-assisted development tools, as they are able to understand the structural and semantic relationships that appear in the source code. These approaches enable automated systems to be able to give context-aware feedback instead of only looking at predefined rules. So, the integration of AI into the software development environment has received significant attention as a way of enhancing developer productivity and software quality. Intelligent automation systems are becoming more widely investigated for purposes such as bug detection, vulnerability detection, summarizing code, automated code review. By leveraging data-driven learning techniques along with conventional static analysis methods, these types of systems can provide more accurate and meaningful insights to developers as well as mitigating the cognitive load of the manual examination processes.

There are recent researches that are focused on the development of AI enhanced tools that can help developers during the code review process. These tools are focused on automating dull tasks and giving a way to the feedback that is actionable in right inside the development surroundings. AI-based code review assistants examine the source code for syntactic, semantic, and logical errors as well as improvements that can be made for better maintainability and readability of code. Some systems use intelligent agents that can analyze code changes and create some kind of automated output of feedback like that of human reviewers. Also, natural language processing methods used for interpreting developer comments, documentation, commit messages to provide systems with more relevant suggestions in a code analysis. Real-time code analysis systems have also been considered in order to find programming errors during development, thereby allowing developers to achieve immediate feedback while developing the codes. Such systems attempt to shorten the time spent debugging and increase the number of efficient times a developer spends working on the project, since by finding problems in earlier stages of the coding process. The combination of AI-powered analysis tools with modern development environments further allows seamless interaction between developers and intelligent feedback help system that will make sure that the automated feedback is delivered without interrupting the developer workflow.

In spite of the achievements in automated software analysis and in intelligent development tools, there are a number of issues left to solve when it comes to systems that are able to provide effective support to a real-life development practice. Traditional static analysis tools can generate a lot of warnings, many of which can need to be manually filtered before they become useful, and in that case their usefulness can be diminished. At the same time, purely machine learning-based systems can be challenged due to their inability to interpret and be reliable when applied to complex software projects. To overcome these challenges, hybrid methods that use rule-based analysis in combination with machine learning techniques have been suggested as a promising solution to these challenges. Such approaches take advantage of the precision of static analysis tooling and add to this the context of the understanding provided from AI models. By combining a wide range of analysis methods in a single framework, automated code review systems would be able to provide more accurate and pertinent feedback to developers. Furthermore, the link these systems with the development environments will also enable developers to receive automated insights right during the coding process. This approach helps facilitate constant feedback, allowing developers to detect potential issues at an early stage and have a better remit to maintain the quality of software at high standards throughout the development lifecycle. As research in intelligent software engineering has developed over the years, we can expect AI-powered development assistants to have a major role in enhancing productivity, ensuring code reliability, and boosting efficiency in software development as a whole.

II. LITERATURE SURVEY

A. Code Review Systems with the help of AI

Artificial intelligence has seen wider use to automate and complement code reviews in the contemporary software development environment. AI-powered systems analyze source code to find the defects, inconsistencies of style and potential vulnerabilities in software code while generating automated feedback for developers. These systems aim to lessen the amount of time spent manually inspecting and consistency of the review process. Intelligent code review assistants have been made to offer automated suggestions and indicate problematic code patterns using machine learning models that are trained on vast repositories of code. Such approaches allow developers to get early feedback during development that will help increase the quality and maintainability of the software. The introduction of AI into the code review workflow has shown the potential to greatly improve not only development efficiency [1], but also code reliability [3], [12].

B. Machine Learning in Automatization of Softwares

Machine learning has been used in a way that has changed software engineering in autumn that are

defect detection, code quality, and the efficiency of developer. By analyzing historical code data and software repositories, machine learning models can learn patterns that are associated with general errors, the best practices with respect to programming. These capabilities allow automated systems to understand or find anomalies and give predictive knowledge about possible defects. Intelligent automation frameworks have been discussed to help developers to maintain the quality of the software while reducing manual adhering work. The use of machine learning algorithms in software engineering has thus helped the development of smart tools which can assist developers during the software' life cycle [2], [7], [8].

C. Natural Language Processing of Code Analysis

Natural language processing techniques have been used more recently for analyzing and interpreting the programming code and developer documentation. NLP-based systems provide the functionality of analyzing the structures of source codes in addition to textual data (comments, documentation, and commit messages). This ability of intelligent tools to explain selectively and recuperatively, even in a code review session, is part of our efforts to give intelligent tools the ability to provide contextual insights and automated explanations during code review. NLP-driven code review systems can find logical inconsistencies and make suggestions to make the code more readable and maintainable. These techniques aid automated feedback mechanisms that help developers to better understand the issues in the code. The role of NLP methods integrated into tools of software engineering has therefore enhanced the capacity of an automated assistant to provide meaningful and context-aware development assistance [4], [12].

D. Real Time Error Detection and Automated Bug Detection

Recent research has placed a particular importance on real-time code analyzing systems that find errors in programming during the course of the programming process. These types of systems analyze code at a syntactic and semantic level to detect logical errors, runtime risks, and security flaws before the application of code is executed or deployed. Automated bug detection frameworks have been made up to track code round-the-clock and offer suggestions for possible bug fixes. Such approaches try to reduce time needed for debugging and increase the reliability of the software products by spotting issues at an earlier stage of the development lifecycle. Intelligent systems that can detect error over multiple programming languages have shown promising results to support efficient and proactive software development practices [5], [8], [13].

E. Hybrid Artificial Intelligence Architectures for Software Quality Assurance

Hybrid intelligent systems that differentiate traditional static analysis techniques with advanced machine learning approaches have been suggested to increase the effectiveness of automated software analysis tools. Static analysis techniques have the advantage of rule-based identification of code quality problems; machine learning models, providing contextual knowledge of complex code patterns. By combining the use of different techniques in a unified architecture, automated code analysis systems can give more accurate and meaningful feedback. Hybrid approaches have been pursued to, among other tasks, relating vulnerability detection and code clone detection, and software quality assessment. These architectures harness the advantages of a variety of analytical techniques to build complete development support tools that can help increase the quality of the code and the productivity of the developers [6], [9], [11].

III. DATA SOURCES

The efficacy of an artificial intelligence-based code review assistant is dependent on the availability of different and representative code-related data sets. Large scale public software repositories are useful resources for training and testing automated code analysis systems. Open-Source platforms are those that have huge collections of source codes in many languages, as well as commit histories and development discussions. These repos can allow ML models to know coding patterns, how to fix bugs, and normal behaviors of the software/software development life. By analyzing such data sets, intelligent systems can identify common programming errors and efficiently find out patterns related to the software defect. These resources that are available to the public thus constitute an important foundation for constructing automated code review and developer assistance tools.

In addition to source code repositories, records of pull requests and version control history have a lot of very interesting information about the software development process. Pull requests usually have code changes, reviewer comments, and discussions between developers, and this provides valuable contextual information related to code changes and reasons for improvements. These records enable automated systems to obtain a learning of how human reviewers think about code quality and what issues exist during the collaborative development. Code difference analysis and review feedback combined with machine learning models could learn patterns related to changes that were accepted, changes that were rejected, and common issues that are raised in reviews. Such information is in support

of developing intelligent systems that can be used to create meaningful automated review suggestions.

Bug tracking systems and issue management platforms are also an important source of data for automated code analysis systems. These platforms log defects in software, bug reports, and related code fixes in software, so as to provide marked information about problematic code segments and their fixes. By connecting bug reports to hooks/commits or pull requests, machine learning models can gain knowledge patterns between code relationships and software defect. This information can help automated systems to check for similar issues arising in new code and can help developers know which bugs could be there in their code while developing them. The integration of the issue tracking data therefore increases the ability of AI-based tools to deliver accurate detection of defect and code quality analysis.

Another good source of information for automated code review systems are static analysis reports written by existing software quality tools. These reports include detailed descriptions of code violations, security warnings and complexity measurements that can be used to help uncover problematic code structures. Such information can be used as training signals to machine learning models or can also be used as complementary inputs for hybrid analysis frameworks. In addition, there are developer knowledge sources like technical documentation and programming discussion platforms that an individual can use to understand common programming challenges and best practices. By using information from these different sources of data, AI-based code review assistants are able to gain a deeper understanding of software development patterns and feedbacks to provide to developers in an automated manner.

IV. SYSTEM STRUCTURE

The proposed system is formulated in the form of modular architecture combining auto-static code analysis and AI-based contextual understanding of code to help developers during the code review process. The system structure separates developer interaction aspect, analysis processing, and feedback generation aspect to ensure scalability, flexibility, and efficient processing of the source code. This modular design can let different components of the system run separately but have coordinated communication over the analysis pipeline. The way this system works is that the source code is sent to the system from the developer's development environment, processed through several analytical modules, and structured feedback is returned to the developer with detected issues and suggestions for improvement.

At the front-end level, the system interfaces with the developers via IDE-integrated interface. This interface is used by the developers to submit the source code for analysis from the development environment itself. The IDE interface is the door into the system and does the task of relaying the source code to the processing modules on the back end through an application programming interface. The in-doing integration of the review assistant with the development environment assures the developers get feedback without leaving their coding workspace and avoid context switching to boost efficiency in software development. The interface also shows analysis results in a structured way such as warnings, found bugs and suggestions of improvements generated from the analysis pipeline.

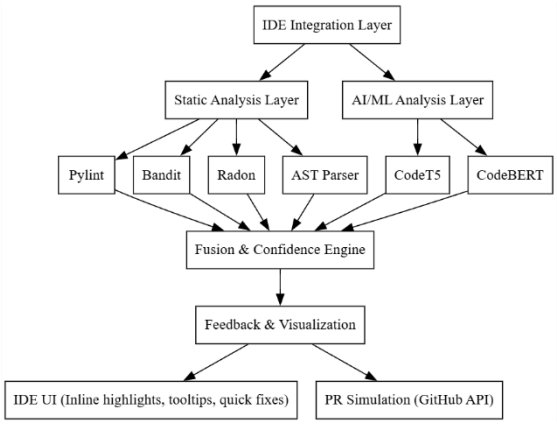

***Fig. 1**. System architecture of the AI-based code review and pull-request assistant.*

The backend processing layer forms the heart of the system and is the reason for the execution of automated code analysis. This layer is made up of several analysis modules that work in a common review pipeline. The pipeline is responsible for coordinated execution of analysis things like rule-based static analysis tools and artificial intelligence-based analysis models. Static analysis modules check the submitted code for syntax errors, violations of the quality of the code, and complexity or potential security violations by employing known software analysis techniques. In parallel, analysis modules based on AI also assess the contextual structure of the code, also finding deeper logical problems and potential improvements. By running both mechanisms of the analysis at the same time, the system will guarantee the cosmic evaluation of a submitted source code.

A central analysis engine is used to control the interaction between the static analysis tools and AI-based models. The outputs generated by these modules are gathered and processed by fusion mechanism which considers the priorities for detected issues and filtering redundancy of warnings. This component guaranteed that the feedback delivered to the developer is brief, pertinent and guiding. After the results of the analysis are processed, the system produces structured feedback information that includes a list

of detected issues, explanations and suggestions for improvement. The results are then sent back to the IDE interface where they are displayed to the developer, through visual indicators, suggestions, and comments on review of results. This predefined system architecture helps the automated assistant to assist the developers by rendering in-time code analysis and giving a rise to the overall quality of the software in the development process.

V. PROPOSED METHODOLOGY

A. Input of code and interaction with IDE

The propose methodology starts with the developer interacting with the system using a integrated development environment. The developer writes or submits source code into the development interface which serves as the gateway for the automated review system. Once the code is submitted, it is passed to the backend analysis system via an application interface which controls the communication between the development environment and the processing modules. This step is used to make the system possible to receive code snippets or files directly from the development workspace. The IDE interaction layer enables seamless integration between the developer workflow and the automated analysis pipeline which ensures that code review assistance takes place without interrupting the development.

B. Pipeline Initialization Review

The submitted code is then sent to a centralized review pipeline responsible for orchestrating the analysis process after it has been received by the system. The pipeline is and organizational structure that maps the series of operations needed to analyze the code and allot tasks to the various analysis modules. This much step makes it sure that the incoming code is appropriately ready to be analysed by the static analysis tools and the AI-based analysis components. The review pipeline oversees the processing processing flow, invokes each of the modules in the correct order, and works, collect intermediate outputs that are produced not up to the analysis process. By keeping the processing pipeline in a structured manner, the system guarantees uniform processing of the developer's input and organisation between multiple techniques for multiple analyses.

Fig. 2. *Workflow of the AI-based automated code review process.*

C. Static Code Analysis

The static analysis phase is used to evaluate the submitted code using a rule entails software analysis tools. This module examines the very structure of the code to look for syntax errors, coding style offenses, problems with complexity in the code, and potential areas of security vulnerability. Static analysis tools analyze the code without running it, as they allow for detecting common issues in programming at an early stage. The analysis process detects issues such as unused variables, undefined references, formatting inconsistencies and other rule-based violations that would impact on the quality of a code. The results produced by the static analysis module offer a first layer of automated code review, meaning that the system is able to detect structural and stylistic problems quickly before going further with the deeper contextual analysis.

D. AI-Based Code Analysis

Following static analysis, the system does AI-based analysis to analyze the contextual structure and semantics of the submitted code. This stage uses machine learning algorithms that have been constructed on intelligence to make sense of programming patterns and code relationships. By examining the structure of the code, the AI component is able to determine the deep logical problems and points for improvement that might not be detected by checking rules. The models assess patterns regarding the quality of code, bugs that might appear in code, and maintainability issues. AI - based analysis makes the system more powerful to give intelligent feedback by finding the patterns learned from the data related to software development and giving insights to the developers for his better code.

E. Fusion and Result Processing

After both of the stages of analysis are over, the outputs from the static analysis tools and the AI models are gathered and processed through a mechanism created by fusing them together. This component brings together into a single unit the results produced by the various analysis modules and prioritizes the issues detected according to their relevance and severity. The fusion stage also includes the filtering of duplicate warnings and structuring of the output in the analysis into structured feedback. Combining a rule-based detection approach and AI-generated insights, the system helps the developer get meaningful and succinct recommendations. This combination of various outputs from the analysis results in improved accuracy of automated feedback and makes the review system in general more effective.

F. Feedback Generation & Developer Output

The last stage of the methodology is the generation of structured feedback and this feedback is delivered to the developer via the development interface. The prioritized results of the analysis are formatted by the system in warnings, suggestions and improvement recommendations that can be easily interpreted by the developer. These outputs are presented directly in the development environment (and allow the developers to detect any problems and apply corrections without leaving their coding environment). The feedback generation module makes sure that detected problems are clearly and efficiently conveyed. By delivering overturn immediately automated insights through the course of development, the technique helps to help good code high-quality and extra efficient software program improvement workflows.

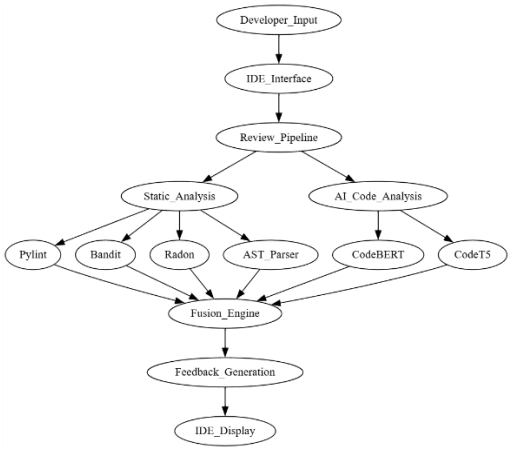

Fig. 3. *Hybrid static and AI-based code analysis pipeline.*

ALGORITHM

Automated AI Based Code Review Process

Step 1: Bootstrapping automated code review system and setting up backend analysis environment.

Step 2: The developer writes or pastes source code from the interface of integrated development environment.

Step 3: The IDE interface takes the code that is submitted and forwards the source code request to the backend API.

Step 4: The backend API receives the request and initializes the review pipeline that is responsible for the coordination of the analysis process.

Step 5: The review pipeline imposes validity and preparation of submitted source code for automated analysis purposes.

Step 6: The pipeline passes the code to a static analysis module, which inspects the code for violations of various coding-rule based on a series of rules.

Step 7: The static analysis module addresses the code structure and identifies problems such as syntax errors, formatting issues, variable use, code complexity as well as potential security problems.

Step 8: The outcomes produced by the static analysis tools are stored as preliminary analysis output.

Step 9: The review pipeline also sends the source code to the AI-based analysis module at the same time.

Step 10: The AI analysis module analyses the code patterns and looks for logical problems, opportunities for code improvements, and maintainability problems.

Step 11: The outputs from both static analysis and analysis based on AI are collected by fusion engine.

Step 12: The results of the fusion engine are processed, duplicate warnings are removed, and issues are ranked by relevance and severity.

Step 13: The processed results are transformed into structured feedback with warnings, found problems and suggestions for improvement on the system.

Step 14: The created feedback is pushed back to the IDE-interface through the backend system.

Step 15: The IDE interface shows the results to the developer in the form of review comments, warnings and improvements that can be suggested to correct code.

VI. RESULTS AND DISCUSSION

A. System Output Observation

The experimental output of the proposed automated code review system shows the capability

of detecting common programming issues during the code analysis. When source code is submitted via development interface, input is passed through the review pipeline, by using the integrated analysis modules. The system generates structured feedback about possible code quality issues such as formatting inconsistencies, variables not used, undefined variables and violations of a naming convention. The feedback given is shown to the developer in the interface in the form of warnings and suggestions. These observations suggest that the static analysis aspect of the system manages to perform baseline code quality evaluation successfully and give developers immediate insight of potential coding issues.

B. Results of Static Analysis Detection

The static analysis layer creates the currently functioning component of the system and gives the main analysis results. This layer checks the structure of the code and programming practices by means of the rule-based detection techniques. The problems are identified in relation to code formatting, variable usage and consistency of the code. The system occurs warnings whenever the submitted code breaches predefined coding standards. These outputs confirm the ability of the rule-based detection process to detect common programming mistakes without having to execute the code. The results generated from the static analysis stage serve as the starting point for the automated review pipeline and showcase the feasibility to have automated quality checks in the development environment.

C. Integration Status of Analysis using AI

The integration stage of the development of the AI-based code analysis module is currently in progress. The role of this module is to analyse the occurrence of code pattern in context and detect logical issues deeper than can be detected by rule based checks only. Although the AI analysis layer is not fully functional at the current stage, with the integration of the layer in the architecture, the system can scale beyond static analysis capabilities. The expected functionality includes identifying complex patterns in the programming code, seeing potential improvement and giving context context. Once fully integrated, this module will work in conjunction with the results of the static analysis and form part of more rounded automated code review functionality.

D. Feedback Creation and Interpreting of Outputs

The system generates feedback by structuring issues that are detected into the output that has a structure and is interpretable by developers. The feedback mechanism for warning and improving the course of development can be presented directly inside the development interface. These outputs help the developers to detect coding issues in a short period of time and to further correct the problems during the development process. The feedback structure is intended to make things clear and relevant, by indicating particular places in the code with found problems. This approach helps developers to understand the problems detected efficiently and would contribute to the improvement of code quality in a path. The feedback generation stage is therefore the last stage in the creation of actionable insights from analysis to drive development.

E. Identified Code Issue Categories

Table I
Detected Code Issue Categories

Issue Category	Description	Detection Source
Formatting Issues	Code formatting inconsistencies such as trailing whitespace	Static Analysis
Variable Usage Issues	Unused or undefined variables within the code	Static Analysis
Naming Convention Issues	Variables or constants not following standard naming rules	Static Analysis
Code Structure Warnings	General structural problems affecting code readability	Static Analysis

The categories of issue that were observed show that the current system is effective at finding structural and stylistic code issues that regularly appear during software development.

F. Functional Components Evaluation of the System

Table II
Current Implementation Status of System Components

System Component	Function	Current Status
IDE Interface	Accepts source code input from developer	Implemented
Review Pipeline	Coordinates code analysis process	Implemented

System Component	Function	Current Status
Static Analysis Module	Detects syntax and structural issues	Implemented
AI Analysis Module	Performs contextual code analysis	Under Integration
Fusion Engine	Combines outputs from analysis modules	Partial Implementation
Feedback Generation	Displays warnings and suggestions	Implemented

This evaluation highlights the fact that the system architecture is implemented partially with core components, which are already operational.

G Discussion of Observed System Behavior

The behavior of the system observed shows the practical ability of automated code analysis in the development environment. The currently implemented modules are successful in detecting several common programming issues and putting them in a structured form for a developer to see. The results show that the integration of automated analysis in the development process can help developers to ensure that coding standards are followed and potential issues can be detected early in the development process. Although the AI-based analysis part is still being integrated, the now built system structure offers a good basis for expanding the analysis capabilities. Future development will be to complete the integration of the AI models to improve the contextual understanding ability and increase the richness of the automated code review feedback.

VII. FUTURE WORK

The proposed AI-based code review and pull request assistant aims at establishing a foundation for automated code analysis in an integrated development environment. However, a number of improvements are planned in order to expand the capabilities of the system and make it more effective in real development scenarios. One major direction for future work will be on the full integration of the AI-based analysis module. While the current implementation great focus is on the rule-based static analysis, future versions will include machine learning models that are capable of analyzing the contextual programming patterns and identifying deeper logical issues in the source code. This enhancement will allow the system to give more intelligent feedback than syntax error detection and include more advanced code quality analysis.

Another significant area for improvement is to strengthen the mechanism for fusing the outputs from the static analysis tools and AI-based models. Future work will focus on getting better at prioritizing and filtering the issues detected so that developers get short and highly relevant feedback. By improving the refinement of the fusion process the system will be able to minimize redundant warnings and emphasize the most critical problems that need to be addressed immediately. Additional improvements to the feedback generation module will also be investigated to give a clearer explanation and context and more structured recommendations. These improvements will help developers understand the issues detected by the system better and make corrections in a faster way in the development process.

Expanding the system to require less language-specific behaviors and support a special enough set of programming languages is another major improvement in the future. The current implementation is geared towards analyzing one type of source code, but in modern software systems, there is often several programming languages and frameworks. Future versions of the system would seek to expand compatibility to more types of programming environment and let the automated assistant assist with diverse software projects. In addition, deeper integration with version control systems will enable the assistant to use the version control system to automatically analyze pull requests and generate review comments.

Further research will also focus on enhancing the system's capacity as assistive tools for the developer during continuous development activities. Integration with automated testing frameworks and development pipelines can support the system to execute code analysis in a manner which is part of the continuous integration. This ability will enable developers to get automated feedback at early stages of the software development and limit the chances of defects moving to later stages in development. Improving the real-time analysis abilities will also make the system more responsive and give developers immediate suggestions while working on the code.

Another important direction for future work is to conduct extensive experimental evaluations and user studies to evaluate the effectiveness of the automated code review assistant. These evaluations will measure such factors as issue detection capability, usefulness of feedback, and developer interaction with the system. User studies with developers can give insights on the impact of the automated assistant on coding practices, development efficiency and software quality. The

results obtained from these evaluations will be used to guide improvements in the system architecture, analysis pipeline and feedback presentation mechanisms.

Finally, there should be some enhancements in the future which understands more about intelligent feature, such as automucing code refacturing suggestion, generating documentation for some context, improved explanation mechanism of some detected issue. These capabilities can turn the automated assistant into more round development support system that has the capacity to not only identify issues but also to help developers in understanding and resolving them successfully in an effective way. By continuously solving how to improve the integration of artificial intelligence techniques and software analysis techniques, the proposed system can become a strong auxiliator in the modern software engineering practice and contribute to the elevation of the quality of code and the productivity of software development.

CONCLUSION

This study worked around the limits of the traditional manual code review process which is not only time consuming and inconsistent but also prone to human error. In order to overcome these difficulties an AI-based code review and pull-request-assistant was put forward to automate code quality analysis within a development environment. The system combines the rule-based static analysis methods and a framework for analysis integrated through an AI to perform a structured review pipeline. This type of architecture allows this system to analyze source code and identify some common programming issues, producing structured feedback to help developers during the development process.

The proposed system adds a modular system that includes the interaction of IDE, coordination of the automatic review pipeline, static analysis modules, integration of AI-based analysis and feedback generation mechanism. Experimental observations show that the implemented components have been successful at identifying common problems with the quality of the code and clearly give developers actionable feedback directly in the development interface.

Although the implementation of the AI analysis part is still in the integration phase, the current implementation shows the feasibility of automated code review systems in supporting software quality assurance. The proposed framework lays the groundwork for smart developer assistive tools that can help improve code reliability, the code review process, and aid efficient software development settings.

REFERENCES

[1] H. Hiththatiyage and N. Kodikara, "AI-Enhanced Code Review Assistant for Web Developers: Designing, Implementing, and Evaluating an AI-Powered Web Application for Automated Code Feedback," 2026.

[2] E. N. Noor, N. A. Malik, M. Zubair, A. Safi, L. Gul, M. H. Khan, and S. N. Batool, "Intelligent Automation in Software Engineering: Transforming Developer Productivity and Code Quality Through Machine Learning," *Spectrum of Engineering Sciences*, vol. 3, no. 12, pp. 492–514, 2025.

[3] R. Konda, "AI-Powered Code Review: Enhancing Software Quality with Intelligent Agents," *IJLRP – International Journal of Leading Research Publication*, vol. 4, no. 3.

[4] K. Mathew, "Automating Code Review Systems Using Natural Language Processing," *International Journal of Emerging Trends in Computer Science and Information Technology*, pp. 593–603, 2025.

[5] T. Asharaf, S. Thanusri, K. J. Febin, S. Santhosh, and C. Sanjay, "Real-Time Syntactic, Semantic, and Logical Error Detection Using AI in Multilanguage Code Editors," *International Journal of Computational Learning & Intelligence*, vol. 5, no. 2, pp. 981–988, 2026.

[6] A. A. Almatrafi, F. A. Eassa, and S. A. Sharaf, "Hybrid Intelligent Architecture for Context-Driven Code Clone Detection," *Journal of King Abdulaziz University: Computing and Information Technology Sciences*, vol. 14, no. 2, pp. 44–50, 2025.

[7] S. Ahmed, "Integrating AI-Driven Automated Code Review in Agile Development: Benefits, Challenges, and Best Practices," *International Journal of Advanced Engineering, Management and Science*, vol. 11, no. 2, pp. 1–10, 2025.

[8] I. C. Emeto, U. W. Anthony, A. A. Galadima, C. O. Ajayi, A. Fahm-Yesufa, A. M. B. Amad, and D. C. Elenwo, "A Web-Based Automated Bug Detection and Fix Suggestion System," 2025.

[9] M. Moradi, K. Yan, D. Colwell, and R. Asgari, "Artificial Intelligence for Context-Aware Visual Change Detection in Software Test Automation," *Progress in Artificial Intelligence*, pp. 1–19, 2025.

[10] V. Chhetri, K. Upadhyay, A. B. Siddique, and U. Farooq, "What Users Value and Critique: Large-Scale Analysis of User Feedback on AI-Powered Mobile Apps," *arXiv preprint arXiv:2506.10785*, 2025.

[11] S. B. Barma, M. Hariharan, and S. Arvapalli, "Enhancing Software Quality Assurance with an Adaptive Differential Evolution Based Quantum Variational Autoencoder–Transformer Model," *arXiv preprint arXiv:2503.16335*, 2025.

[12] B. T. Scholar, "CodeMedic: An AI-Powered Code Review and Assistance Platform."

[13] S. Lau and P. J. Guo, "The Design Space of LLM-Based AI Coding Assistants: An Analysis of 90 Systems in Academia and Industry," in *Proc. IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pp. 300–313, 2025.

[14] A. Kovalyova, "Controlling Software Code Vulnerabilities Using AI-Oriented Static Analysis," *Measuring and Computing Devices in Technological Processes*, no. 3, pp. 7–11, 2025.